PDF 1 DE 2 · TEORIA · RESOLUÇÃO DE PROBLEMAS ESTRUTURADOS EM COMPUTAÇÃO

Tabela Hash e Ordenação

Todo o conteúdo das aulas (slides *8-Tabela Hash*, *Hotel Hash*, *Material de Apoio*) resumido para quem está começando. Leia este primeiro; depois o PDF 2 (Prática) mostra o projeto da Creche dos Bichos (Cenário B) sendo construído com TDD.

1 O problema: achar um dado rápido

Busca sequencial: olhar posição por posição até achar. Procurando João em {Ana, Pedro, João, Maria, Bruna} são 3 comparações; com 1 milhão de bichinhos, o último custa 1 milhão. O esforço cresce junto com n: é **O(n)**.

Tabela hash (ou de dispersão, de espalhamento): uma **função hash** faz uma conta com a **chave** e devolve um **índice** de 0 a M − 1, a posição do vetor onde o dado fica. Para buscar, faz a **mesma conta** e vai direto à posição: em média **O(1)**, o mesmo esforço para 10 ou 10 milhões de chaves.

> **NA CRECHE** Os canis são numerados. Quando um bichinho chega, a monitora faz uma conta com o **nome** dele e o resultado é o número do canil. No fim da tarde, para buscar a Luna, faz a mesma conta e vai direto ao canil certo.

Termo	Significado
Chave	dado usado na conta (nome normalizado: LUNA)
Índice / balde	posição do vetor / o que fica nela (o canil)
M	quantidade de posições (canis)
Colisão	chaves *diferentes* no mesmo índice
Fator de carga α	chaves ÷ M (bichinhos por canil: a lotação)

2 A função hash

Uma boa função hash (slides): é **sempre a mesma conta** na inserção e na busca, devolve índice de **0 a M − 1**, gera **poucas colisões**, é **rápida** de calcular e **espalha de forma uniforme**. O `%` (resto da divisão) garante o intervalo: o resto da divisão por M só pode ser 0, 1, …, M − 1. Ex.: 304 ÷ 13 = 23 e sobra 5 (13 × 23 = 299), então `304 % 13 = 5`.

Letras − 1 (slides). Ana → 2, João → 3, Pedro → 4. Problemas: sem `% M`, "Gabriel" daria 6, posição que não existe num vetor de 0 a 4 (com `% 5`: 1); e todo nome de 5 letras cai no mesmo lugar (Pedro e Maria colidem em 4).

Soma dos códigos (exigida no projeto). Cada letra é um número (A = 65 … Z = 90). LUNA = 76 + 85 + 78 + 65 = 304 → 304 % 13 = **5**.

```
int soma = 0;
for (int i = 0; i < chave.length(); i++) {
    soma += chave.charAt(i);    // 'L' vale 76...
}
return soma % m;
```

Ponto fraco: a ordem das letras não importa. AMOR e ROMA (303), e também FRIDA e CHICO (358), têm a mesma soma, e por isso colidem em **qualquer** M.

Soma ponderada (uma opção de "função do grupo"). Cada letra vezes a sua posição: LUNA = 76×1 + 85×2 + 78×3 + 65×4 = 740 → 740 % 13 = 12. Agora AMOR (784) ≠ ROMA (731) e FRIDA (1050) ≠ CHICO (1093).

Polinomial (Hotel Hash, igual ao `String.hashCode()`): `h = 31 × h + código`. "Ana": 65 → 2.125 → 65.972; 65.972 % 5 = 2. Em nomes longos, `h` estoura o `int` e fica negativo; por isso usa-se `Math.floorMod(h, M)`, que nunca dá negativo.

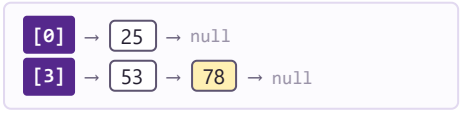
Por que M primo? Chaves 10, 20, 30, 40, 50, 60: com `% 10` todas vão para 0; com `% 12` só para os canis pares; com `% 13` ficam 10, 7, 4, 1, 11, 8, todas diferentes. Se a chave e M têm um divisor em comum, só sobram canis múltiplos dele; um primo não tem divisores, então espalha melhor. **Ajuda, mas não garante**: é preciso testar com os dados reais.

3 Colisões e as três estratégias

Colisão é quando **chaves diferentes** dão o mesmo índice. É inevitável: 50 bichinhos em 13 canis obrigam a dividir. Não se joga ninguém fora; é preciso guardar os dois e achar os dois depois. Atenção: **pets com o mesmo nome** (três Lunas) têm a **mesma chave**, então caem sempre no mesmo canil, qualquer que seja M. Isso não é colisão no sentido técnico. LUNA (304) e THOR (317) com M = 13 (as duas dão 5), sim, é colisão.

Encadeamento exterior (o do projeto)

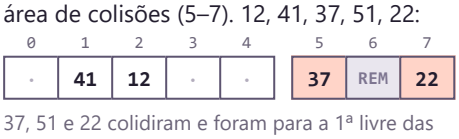
Cada posição guarda uma **lista ligada**; quem colide entra na lista. Slides, `% 5`: 25 → [0]; 53 e 78 → [3].



Busca 78: vai a [3] e percorre só essa lista (2 comparações). Remoção: tira o nó da lista. Pior caso: todos na mesma lista → O(n). Aceita mais chaves que posições (α > 1).

Endereçamento aberto heterogêneo

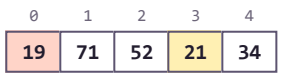
Um vetor só: área de inserção (0–4, `% 5`) e área de colisões (5–7). 12, 41, 37, 51, 22:



37, 51 e 22 colidiram e foram para a 1ª livre das colisões. Busca 37: [2] tem 12, então percorre a área de colisões até achar ou até um **vazio**. Se a área encher, a inserção falha.

Homogêneo (teste linear)

Sem área reservada: colidiu, tenta a próxima posição, `(h + i) % N`, dando a volta no fim. 71, 52, 21, 34, 19 com N = 5:



21 % 5 = 1: [1] e [2] ocupados → [3]. 19 % 5 = 4: ocupado → (4 + 1) % 5 = 0. Ponto fraco: **agrupamento** (blocos contínuos crescem).

REMOÇÃO NO ENDEREÇAMENTO ABERTO A busca para no primeiro **vazio** ("daqui para frente nunca entrou nada"). Se a remoção do 51 deixasse a posição 6 vazia (`null`), a busca do 22 pararia ali e diria "não existe". Por isso a posição removida recebe uma marca (−1 nos slides, `REMOVIDO` no Hotel Hash): a busca **pula** o removido e só para no vazio de verdade.

4 Lista ligada feita à mão

Cada **nó** guarda um dado e uma seta para o próximo; o canil guarda só a seta para o primeiro; o último aponta para `null`.

```
class No { Pet pet; No proximo; }
// percorrer o canil i
No atual = baldes[i];
while (atual != null) { ...; atual = atual.proximo; }
// inserir no início (nessa ordem!)
novo.proximo = baldes[i];  // 1) novo -> antigo 1º
baldes[i] = novo;          // 2) canil -> novo
// remover: "costurar" a lista por cima do nó
if (anterior == null) baldes[i] = atual.proximo;
else anterior.proximo = atual.proximo;
```

Inverter os passos da inserção perde o resto da lista. Inserir no início é O(1); os slides inserem no fim (o 78 depois do 53), e as duas formas são corretas.

5 Fator de carga e rehashing

α = n / M. No encadeamento, é o tamanho médio das listas, ou seja, quantos bichinhos se olha numa busca (na creche, é a **lotação**). 20 pets em M = 13: α = 1,54. 10.000 pets em M = 17: α ≈ 588 (busca quase sequencial).

Limite de inserção = capacidade × fator de carga (slides): 5 × 0,75 = 3,75, então cabem 3 chaves. O ideal fica perto de 0,7–0,75: perto de 1 sobram colisões; perto de 0 sobra memória.

Rehashing: ao passar do limite, **dobra** o vetor e **reinsere tudo**, recalculando os índices (a conta depende de M). Hotel Hash: 25, 53, 78 em M = 5 dão 0, 3, 3; ao chegar o 10, vai para M = 10 e fica 25 → 5, 53 → 3, 78 → 8, 10 → 0 (a colisão some). Copiar para as mesmas posições seria erro: a nova conta procuraria em outro lugar.

Estatísticas de uma tabela: fator de carga, baldes vazios, maior lista e colisões (para cada balde, tamanho − 1; ou, contando só nomes diferentes, distintos − 1).

6 Ordenação: o que contar e como o trabalho cresce

- **Comparação**: olhar dois valores e decidir qual é maior. **Troca** (*swap*): dois elementos trocam de posição usando uma variável auxiliar (`aux = v[a]; v[a] = v[b]; v[b] = aux;`). **Deslocamento**: um elemento anda uma casa (Insertion).
- **Inversão**: par fora de ordem. {7, 2, 12, 8, 3} tem 5: (7,2) (7,3) (12,8) (12,3) (8,3). Ordenado: 0. Inverso: n(n−1)/2. Aleatório: ≈ n(n−1)/4. Cada troca do Bubble e cada deslocamento do Insertion conserta exatamente 1 inversão.
- Para comparar algoritmos de forma justa, cada um recebe uma **cópia** dos mesmos dados. Variável de vetor guarda uma *seta*: `b = a` não copia; é preciso criar um vetor novo e copiar posição por posição.

O grande diz como o trabalho cresce com n. O(n^2): dobrou n, quadruplica. O(n log n): pouco mais que n.

n	$n \cdot \log_2 n$	n(n−1)/2	n^2
50	282	1.225	2.500
1.000	9.966	499.500	1.000.000
10.000	132.877	49.995.000	100.000.000

Melhor, médio e pior caso dependem da arrumação inicial: é por isso que a Grande Corrida testa aleatório, já ordenado e ordem inversa.

7 Bubble Sort

Compara **vizinhos** (j e j+1) e troca se estiverem fora de ordem. A cada passagem, o maior que falta "borbulha" até o fim; a próxima passagem para uma casa antes. Rastreio de {7, 2, 12, 8, 3}:

Passagem	Comparações (T = troca)	Vetor
1	7>2 T · 7>12 · 12>8 T · 12>3 T	2 7 8 3 **12**
2	2>7 · 7>8 · 8>3 T	2 7 3 **8 12**
3	2>7 · 7>3 T	2 3 **7 8 12**
4	2>3 (nenhuma troca → para)	**2 3 7 8 12**

Total: 10 comparações (4+3+2+1) e 5 trocas (as 5 inversões).

```
for (int i = 0; i < n - 1; i++) {
    boolean houveTroca = false;
    // os últimos i já estão no lugar
    for (int j = 0; j < n - 1 - i; j++) {
        if (v[j] > v[j + 1]) {
            trocar(v, j, j + 1);
            houveTroca = true;
        }
    }
    if (!houveTroca) break;  // sem troca: ordenado
}
```

Já ordenado (com a parada): n − 1 comparações, 0 trocas → **O(n)**. Inverso: n(n−1)/2 comparações e trocas → **O(n^2)**.

8 Insertion Sort

Como cartas na mão: a parte esquerda já está ordenada; pega a próxima (a **chave**), **desloca** para a direita os maiores que ela e encaixa a chave no buraco. Rastreio de {7, 2, 12, 8, 3}:

Chave	O que acontece	Comp	Desl	Vetor
2	7 desliza; acabou	1	1	**2 7** 12 8 3
12	7 > 12? não	1	0	**2 7 12** 8 3
8	12 desliza; 7 > 8? não	2	1	**2 7 8 12** 3
3	12, 8, 7 deslizam; 2 > 3? não	4	3	**2 3 7 8 12**

```
for (int i = 1; i < v.length; i++) {
    int chave = v[i];
    int j = i - 1;
    while (j >= 0 && v[j] > chave) {
        v[j + 1] = v[j];     // deslocamento
        j--;
    }
    v[j + 1] = chave;        // encaixa no buraco
}
```

`j >= 0` vem antes por causa do curto-circuito do `&&` (evita ler `v[-1]`). Deslocamento ≠ troca: 1 atribuição contra 3. Já ordenado: n − 1 comparações, 0 deslocamentos → **O(n)**. Inverso: n(n−1)/2 → **O(n^2)**.

9 Quick Sort (dividir e conquistar)

1) Escolhe um **pivô** (nos slides, o **último**). 2) **Particiona**: menores à esquerda, maiores à direita, pivô no meio, na **posição definitiva**. 3) Repete em cada lado. Isso é **recursão**: o método chama a si mesmo para pedaços menores, até o **caso base** (0 ou 1 elemento). Cada chamada pendente fica numa pilha de memória; empilhar fundo demais causa `StackOverflowError`.

Partição de {7, 1, 3, 5, 2, 6, 4}, pivô 4 (`i` = fim da área dos menores, começa em −1):

j	v[j] < 4?	ação	vetor
0	7 não	—	7 1 3 5 2 6 [4]
1	1 sim	i=0, troca v0↔v1	1 7 3 5 2 6 [4]
2	3 sim	i=1, troca v1↔v2	1 3 7 5 2 6 [4]
3, 4	5 não · 2 sim	i=2, troca v2↔v4	1 3 2 5 7 6 [4]
fim	6 não	pivô ↔ v[i+1]	1 3 2 **4** 7 6 5

Depois: {1,3,2} (pivô 2) e {7,6,5} (pivô 5, depois {6,7}). Total: 11 comparações e 6 trocas (não conta trocar um elemento com ele mesmo).

```java
void quickSort(int[] v, int ini, int fim) {
    if (ini < fim) {              // senão: caso base
        int p = particionar(v, ini, fim);
        quickSort(v, ini, p - 1); // menores
        quickSort(v, p + 1, fim); // maiores
    }
}
int particionar(int[] v, int ini, int fim) {
    int pivo = v[fim], i = ini - 1;
    for (int j = ini; j < fim; j++)
        if (v[j] < pivo) { i++; trocar(v, i, j); }
    trocar(v, i + 1, fim);
    return i + 1;
}
```

Caso médio O(n log n): o pivô divide mais ou menos ao meio → ~$\log_2 n$ níveis × ~n comparações. **Pior caso O(n^2)**: vetor **já ordenado** (ou inverso) com pivô = último. O pivô é sempre o maior (ou o menor), a partição sobra com n − 1 elementos de um lado e nada do outro: n(n−1)/2 comparações e recursão com n níveis (com 10.000, risco de `StackOverflowError`). Evita-se escolhendo o pivô do meio, sorteado ou a mediana de três.

10 Os três lado a lado

Algoritmo	Melhor	Médio	Pior
Bubble (com parada)	O(n) ordenado	O(n^2)	O(n^2) inverso
Insertion	O(n) ordenado	O(n^2)	O(n^2) inverso
Quick (pivô último)	O(n log n)	O(n log n)	O(n^2) ordenado/inverso

Contagens exatas	Já ordenado	Ordem inversa
Bubble	n−1 comp · 0 trocas	n(n−1)/2 comp · n(n−1)/2 trocas
Insertion	n−1 comp · 0 desl	n(n−1)/2 comp · n(n−1)/2 desl
Quick	n(n−1)/2 comp · 0 trocas	n(n−1)/2 comp · n/2 trocas

11 Medindo tempo de forma justa

```java
long inicio = System.nanoTime();
ordenar(copia);
long ns = System.nanoTime() - inicio;
double ms = ns / 1_000_000.0;
```

- **Mesma entrada** para os três (cópias) e **só a ordenação** dentro do cronômetro.
- **Não imprimir** durante a medição: escrever na tela é milhares de vezes mais lento que comparar.
- **Aquecer** (o Java acelera o código depois de rodá-lo algumas vezes, o compilador JIT) e **repetir** tirando a média.
- Com 50 elementos tudo leva microssegundos: o tempo é ruído, então confie nas **contagens**.

EM UMA FRASE Hash serve para **achar** pelo nome em O(1), com colisões tratadas por lista ligada; ordenação serve para montar a **fila da saída** por microchip. Bubble e Insertion são O(n^2) (ótimos só em dados já ordenados); Quick é O(n log n) no caso médio, mas cai para O(n^2) com pivô ruim.